

Scalability optimization

Frontend + Backend

Optimization Technique

Handling server load

Scalability
→ handle more no. of users, traffic, transactions increase without hampering application performance

Importance

1. Handle increased workload on traffic
2. If more usage by users, is it providing seamless
3. Is making business growth
4. Future proofing your software as per needs
5. Ensure fault tolerance and availability

Type of scalability

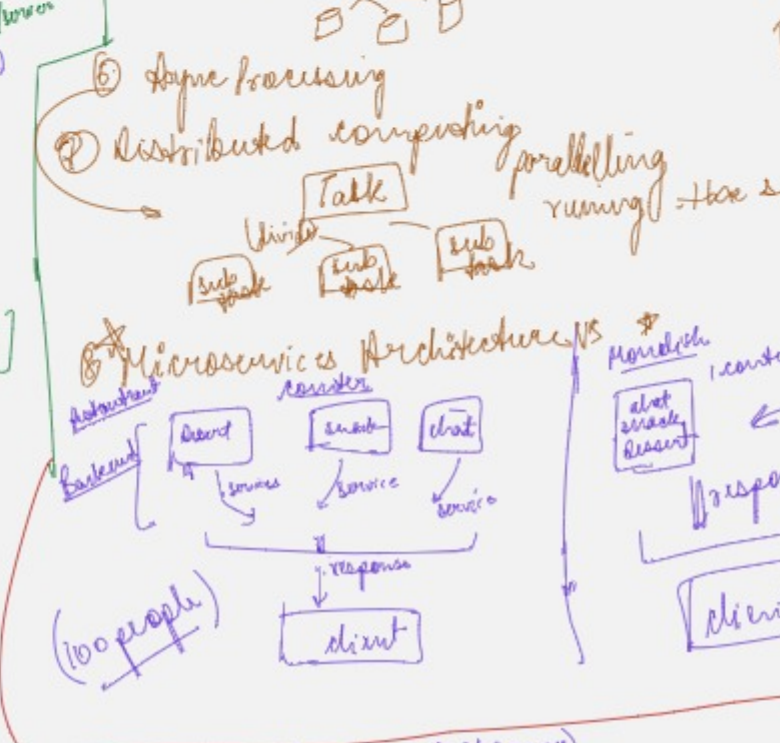
Vertical scalability (scaling up) (single)
→ increasing RAM, storage capacity of a single machine (server)

Horizontal scalability (scaling out) (multiple)
→ adding more machines (servers)

Elastic scalability (auto-scaling)
→ it's a dynamic approach to scale based on resource [either scale up or scale down]

Scalability Technique

1. Load Balancing → handle incoming traffic across multiple servers to efficient utilization
2. Caching → if a request is accessed to reduce the load we will cache the data
3. Elastic Infrastructure → utilizing cloud services as per the demand
4. Sharding → store session state on data, distribute applications
5. Database sharding → partitioning the database into smaller chunks allow for distributed storage and improve scalability



Scalability Testing

1. Load Testing → measure how system is handling increased traffic on workload
2. Stress Testing → try out push your application beyond limit
3. Performance Testing → measure system response time and resource utilization under different loads
4. Capacity Testing → how no. of users or transactions system/application can handle
5. Database Testing → test performance, scalability, infrastructure
6. Cloud based testing → utilizing cloud based testing platform

Testing Tools

- 1. JMeter [load, stress]
- 2. Locust [load testing]
- 3. Loader.io [load testing]
- 4. Tsung [performance]

Scalability on Cloud computing Platforms

1. AWS → AWS auto scaling, elastic load balancing, RDS replicas, ElastiCache, DynamoDB auto scaling
2. Azure → Auto scale, load balancer, SQL DB elastic pools, Redis for Redis, CosmosDB
3. GCP (Google) → Google cloud auto scaling, cloud load balancing, cloud spanner, Redis (memory store)
4. IBM cloud → auto scaling, load balancer, IBM Db2 warehouse, IBM cloudant

Scalability Arch by cloud native Platform

1. Microservices Arch
2. Containerization [put all dependencies parked in container]
3. Event driven
4. Serverless
5. Monitoring & analytics
6. Container orchestration (manage and scale app as per demand)

CDN (Content Delivery Network) → used distributed network of server

- It will load contents based on nearest server as per the user's geographical location
- CD resources are stored in browser cache when you run for first time
- Importance → It will load the web page speedily

Caching → any static content, file, image which is frequently, we will avoid server load we will cache. (CSS files, JS libraries, Bootstrap, Fontash, client, moment.js)

Importance → optimizing the delivery, reducing server load, enhancing/improving performance for end users

Frontend Scalability

1. Performance Optimization → optimize code to minimize load time, optimize storage, minimizing http request, implement caching
2. client side caching → caching techniques to reduce server load and improve response time
3. CDN → use more of CDN
4. Async operations → implement lazy loading
5. Responsive Design

Backend (scalability)

1. Scalable Arch → load balancers, distributed system, caching
2. Horizontal scaling
3. Database optimization → indexing, database queries, caching, sharding
4. Caching
5. Distributed file system